53

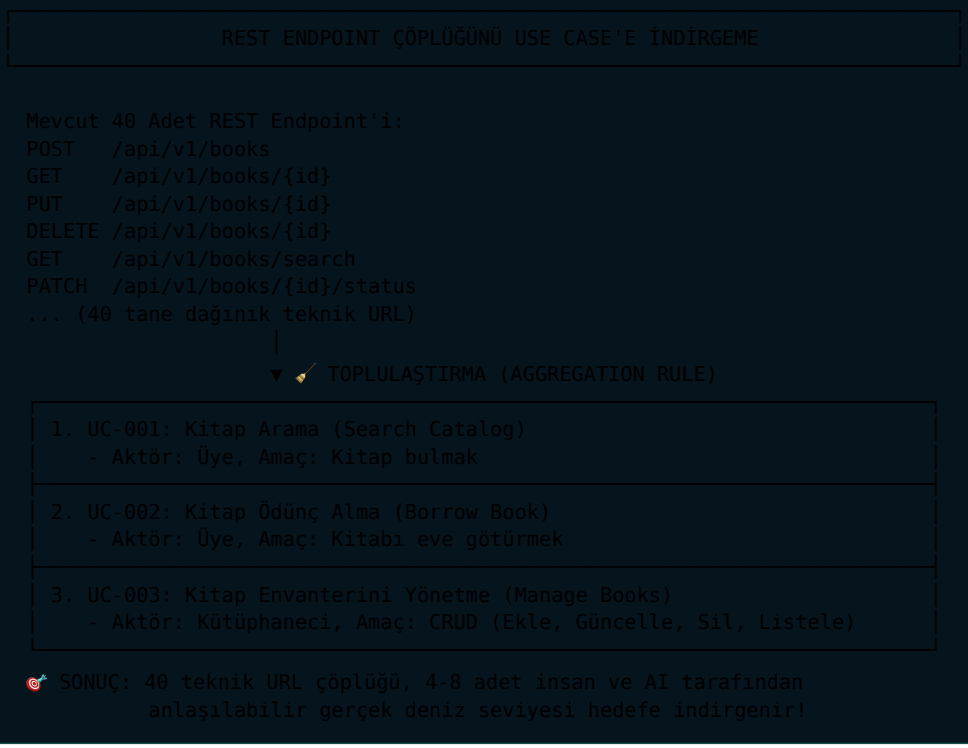
BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Uç Noktaları Birleştirme: 40 REST Endpoint'ini 4-8 Use Case'e İndirgeme

Aggregating Endpoints: Collapsing 40 REST Endpoints into 4-8 Use Cases

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir akıllı telefonun içini açarsanız 500 tane lehimli kablo ve devre görürsünüz. Ama ekranı açtığınızda sadece 4 ikon görürsünüz: Telefon, Mesajlar, Kamera, Ayarlar! Kimse lehim kablosuyla uğraşmak istemez. İşte 40 REST endpoint'i o lehim kablolarıdır; 4-8 use case ise ekrandaki sade ve kullanışlı uygulama ikonlarıdır!

🇬🇧 ENGLISH (ELI5)

If you pry open an iPhone, you find 500 soldered micro-circuits and copper ribbons. But on the screen, you only interact with 4 icons: Phone, Messages, Camera, Settings! Nobody wants to touch raw solder. Forty REST endpoints are internal wiring; 4 to 8 use cases are the clean, intuitive app icons on the home screen!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Endpoint Toplulaştırma ve 'Manage X' Kuralı: Brownfield projelerde yapılan en yaygın hata, her bir Controller metodunu veya REST endpoint'ini bir use case zannetmektir. Bir projede 40 endpoint varsa, 40 tane use case yazmaya kalkan bir ekip şartname çöplüğünde boğulur. Martinelli'nin altın kuralı: 'Controller'lar, rotalar ve ekranlar use case değildir; use case'in delilleridir. Onları toplulaştırın!' 5 farklı CRUD endpoint'i (Create, Read, Update, Delete, List) aynı aktöre ve aynı veri akışına hizmet ettiği sürece tek bir 'Manage Books' use case'idir. Ancak ne zaman bir işlem kendi bağımsız iş kurallarına ve güvenlik rollerine sahip olursa (örn: 'Block Customer'), o zaman o işlem 'Manage X' çatısından ayrılıp kendi eylem seviyesi (action-level) use case'ine dönüşür. Tipik bir mikroservis bu yöntemle 4 ila 8 arası derli toplu use case'e indirgenir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Endpoint Aggregation and the 'Manage X' Heuristic: The fatal pitfall in legacy recovery is mistaking technical controllers or routes for use cases. Documenting 40 use cases for 40 HTTP endpoints yields bureaucratic paralysis. Martinelli's rule: 'Controllers, routes, and views are evidence of use cases, not use cases. Aggregate them.' Five REST endpoints (`POST /books`, `PUT /books/{id}`, `GET /books`, `DELETE /books/{id}`) are consolidated into a single 'Manage Books' use case so long as they share an identical actor and shape. Only when a sub-operation acquires unique preconditions, distinct role-guards, or diverging state transitions (e.g., 'Archive Book' requiring audit approval) does it split out as an action-level use case. A production microservice typically condenses into 4 to 8 user-goal specifications.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 4 REST Endpoint'inin Tek Bir 'ManageBooks' Servisinde Toplulaştırılması
package com.maang.aiup.aggregation;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class EndpointAggregationDemo {
    public record BookRecord(String id, String title, String author) {}

    // Tek bir 'UC-003: Manage Books' use case'i altındaki CRUD operasyonları
    public static class ManageBooksUseCase {
        private final Map<String, BookRecord> db = new ConcurrentHashMap<>();

        public void createBook(BookRecord book) { db.put(book.id(), book); }
        public BookRecord readBook(String id) { return db.get(id); }
        public void updateBook(BookRecord book) { db.put(book.id(), book); }
        public void deleteBook(String id) { db.remove(id); }
    }

    void main() {
        var uc = new ManageBooksUseCase();
        uc.createBook(new BookRecord("B1", "Effective Java", "Joshua Bloch"));
        System.out.println("✅ 4 REST endpoint'i tek bir 'Manage Books' use case'inde
toplulaştırıldı.");
        System.out.println("   -> Okunan Kitap: " + uc.readBook("B1"));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Her HTTP GET veya POST isteği için ayrı bir use case dosyası açmayın! CRUD işlemlerini tek bir 'Manage X' altında toplayın; yalnızca kuralları ayrışan işlemleri split edin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not generate individual use case documents for every HTTP GET or POST route. Aggregate standard CRUD into 'Manage X'; split only when business policies diverge.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- REST uçları use case değildir; use case'in varlığına işaret eden teknik izlerdir.
- Aynı aktör ve akışı paylaşan CRUD işlemleri tek bir 'Manage X' altında toplanır.
- Tipik bir kurumsal mikroservis 40 endpoint'ten 4-8 anlamlı use case'e indirgenir.

🎯 Key Architectural Takeaways

- REST endpoints are not use cases; they are technical evidence of underlying use cases.
- CRUD operations sharing the same actor and flow aggregate into a single 'Manage X' spec.
- A typical microservice condenses from 40 low-level routes into 4 to 8 meaningful use cases.